

Specyfikacja systemu mini-SCADA PID

SPECYFIKACJA TECHNICZNA I ARCHITEKTURA SYSTEMU

Projekt: Serwer Telemetrii i Akwizycji Danych (mini-SCADA) dla regulatora PID

1. Przegląd Architektury (High-Level)

System ma charakter potokowy (pipeline) i składa się z odseparowanych modułów komunikujących się ze sobą w sposób asynchroniczny i bezpieczny wątkowo. Głównym celem jest bezstratny odbiór danych z zewnętrznego regulatora, ich obróbka cyfrowa (DSP), archiwizacja oraz wizualizacja bez blokowania wątku interfejsu użytkownika.

Przepływ danych (Data Flow): [Symulator/Węzeł PID] —UDP→ [Warstwa Sieciowa] —Kolejka MPSC/RingBuffer → [Wątki Przetwarzające DSP] Dispatcher [Baza Danych] & [GUI (Qt)]

2. Moduł Rdzenia: Model Matematyczny PID

Zanim zbudujemy serwer, będziemy potrzebowali wiarygodnego źródła danych. Skonstruujemy matematyczny model obiektu regulacji oraz samego regulatora PID w dziedzinie czasu dyskretnego.

Teoria i wzory matematyczne: Klasyczne równanie regulatora PID w dziedzinie czasu ciągłego wyraża się wzorem:

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de(t)}{dt}$$

gdzie:

- $u(t)$ – sygnał sterujący (Control Output)
- $e(t) = SP - PV$ – uchyb regulacji (Setpoint - Process Variable)
- K_p, K_i, K_d – nastawy regulatora (wzmocnienia: proporcjonalne, całkujące, różniczkujące)

Na potrzeby naszej implementacji algorytmu (Faza 1) będziesz musiał przekształcić to do postaci dyskretniej (np. postaci równania różnicowego), używając czasu próbkowania T_s .

3. Warstwa Sieciowa (Ingress)

Aplikacja musi nasłuchiwać na dedykowanym porcie używając protokołu UDP. Z racji charakteru protokołu UDP, system musi być odporny na zgubione pakiety (Out-of-Order / Packet Loss).

Struktura protokołu binarnego (wyjątek z reguły braku kodu): Poniżej znajduje się ustalamy przez nas

format ramki telemetrycznej. To ten schemat będziesz rzutował z bufora strumienia sieciowego.

```
#pragma pack(push, 1)
struct PIDTelemetryPayload {
    uint32_t packet_id;           // Monotonicznie rosnący identyfikator, do
    detekcji gubienia pakietów
    uint64_t timestamp_ms;       // Czas wygenerowania próbki na węźle (np. epoch
    w milisekundach)
    float setpoint;               // Wartość zadana (SP)
    float process_variable;       // Wartość mierzona (PV) - zaszumiona
    float control_output;         // Wartość sterująca (CV / U)
    float error;                  // Aktualny uchyb (e)
    uint16_t status_flags;        // Flagi bitowe (np. bit 0: ALARM, bit 1:
    MANUAL_MODE)
    uint16_t checksum;           // Suma kontrolna (np. CRC16) upewniająca nas o
    integralności payloadu
};
#pragma pack(pop)
```

4. Przetwarzanie Sygnałów i Wielowątkowość (Warstwa DSP)

Wątek sieciowy **nie może** zajmować się obróbką danych. Jego jedynym zadaniem jest odebranie pakietu i odłożenie go do współbieżnej struktury danych (np. Thread-Safe Queue lub Lock-Free Ring Buffer).

Wątki DSP (Digital Signal Processing): Pule wątków (Worker Threads) będą pobierać surowe pakiety i poddawać je obróbce. Wartość `process_variable` będzie symulować zaszumiony sygnał z czujnika (np. zaszumiony szumem białym). Wątek DSP nałoży na to cyfrowy filtr dolnoprzepustowy (np. prostą średnią kroczącą - SMA, wykładnicze wygładzanie - EMA, lub filtr IIR typu biquad, jeśli zechcesz wykorzystać swoje doświadczenie z obróbką audio).

Wzór na filtrację wykładniczą (EMA), z której prawdopodobnie skorzystamy:

$$PV_{filtered} = \alpha \cdot PV_{raw} + (1 - \alpha) \cdot PV_{previous_filtered}$$

gdzie α to współczynnik wygładzania ($0 < \alpha \leq 1$).

5. Warstwa Składowania Danych (Persystencja)

Przetworzone pakiety muszą trafić do lokalnej lub zdalnej bazy danych (np. SQLite / PostgreSQL).

- **Wymóg wydajnościowy:** Zapis pojedynczych wierszy w czasie rzeczywistym zablokuje I/O. Wymagam zaimplementowania mechanizmu *Batchowania* (grupowania).
- Dane mają być zapisywane do bazy danych za pomocą transakcji, np. w paczkach po 100-500 rekordów lub co określony interwał czasowy (np. co 1 sekundę). Mechanizm ten musi działać w odrębnym wątku, aby nie spowalniać pętli DSP i GUI.

6. Warstwa Prezentacji (Qt GUI)

Interfejs musi działać płynnie, niezależnie od obciążenia bazy danych i liczby pakietów UDP.

- Pomiędzy warstwą C++ (backend) a GUI wdrożysz wzorzec obserwatora / system sygnałów i slotów Qt.
- **Wymagane widoki:**
 - Wykres w czasie rzeczywistym (Time-Series Chart) nakładający na siebie serie: Setpoint, Process Variable (Raw), Process Variable (Filtered).
 - Wskaźniki zegarowe/słupkowe dla uchybu (Error) i sygnału sterującego (Control Output).
 - Tabela logów dla flag alarmowych (dekodowanie status_flags).

Decyzja należy do Ciebie: Czy akceptujesz powyższą specyfikację i ogólny zarys architektury? Jeśli tak, potwierdź, a wyznaczę Ci zakres prac dla **Fazy 1 (Matematyka i logika rdzenia)** i podam wymagania brzegowe do implementacji, po których będę czekał na Twoje pierwsze Pull Requesty / wklejki z kodem do Review.
